FUTURE_DS_01

Business Sales Performance Analytics - Task 1

Step 1.Importing Required libraries

In [2]:
```
import pandas as pd
import numpy as np
```

Step 2.Loading dataset in to our model

In [3]:
```
# Load the dataset (change 'superstore.csv' to your actual file name)
try:
    df = pd.read_csv('superstore.csv', encoding='utf-8')
except UnicodeDecodeError:
    df = pd.read_csv('superstore.csv', encoding='latin1')

# Display the first 5 rows to ensure it loaded correctly
df.head()
```

Out[3]:

	Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Co
0	1	CA-2016-152156	11/8/2016	11/11/2016	Second Class	CG-12520	Claire Gute	Consumer	U
1	2	CA-2016-152156	11/8/2016	11/11/2016	Second Class	CG-12520	Claire Gute	Consumer	U
2	3	CA-2016-138688	6/12/2016	6/16/2016	Second Class	DV-13045	Darrin Van Huff	Corporate	U
3	4	US-2015-108966	10/11/2015	10/18/2015	Standard Class	SO-20335	Sean O'Donnell	Consumer	U
4	5	US-2015-108966	10/11/2015	10/18/2015	Standard Class	SO-20335	Sean O'Donnell	Consumer	U

5 rows × 21 columns

In [4]:
```
df.describe  # for descriptive statistics
```

Out[4]:
```
<bound method NDFrame.describe of       Row ID        Order ID  Order Date   Sh
ip Date       Ship Mode  \
0          1  CA-2016-152156   11/8/2016  11/11/2016    Second Class
1          2  CA-2016-152156   11/8/2016  11/11/2016    Second Class
2          3  CA-2016-138688   6/12/2016   6/16/2016    Second Class
3          4  US-2015-108966  10/11/2015  10/18/2015  Standard Class
4          5  US-2015-108966  10/11/2015  10/18/2015  Standard Class
...      ...             ...         ...         ...             ...
9989    9990  CA-2014-110422   1/21/2014   1/23/2014    Second Class
9990    9991  CA-2017-121258   2/26/2017    3/3/2017  Standard Class
9991    9992  CA-2017-121258   2/26/2017    3/3/2017  Standard Class
9992    9993  CA-2017-121258   2/26/2017    3/3/2017  Standard Class
9993    9994  CA-2017-119914    5/4/2017    5/9/2017    Second Class

     Customer ID     Customer Name    Segment        Country             City
\
0       CG-12520       Claire Gute   Consumer  United States        Henderson
1       CG-12520       Claire Gute   Consumer  United States        Henderson
2       DV-13045   Darrin Van Huff  Corporate  United States      Los Angeles
3       SO-20335    Sean O'Donnell   Consumer  United States  Fort Lauderdale
4       SO-20335    Sean O'Donnell   Consumer  United States  Fort Lauderdale
...          ...               ...        ...           ...              ...
9989    TB-21400  Tom Boeckenhauer   Consumer  United States            Miami
9990    DB-13060       Dave Brooks   Consumer  United States       Costa Mesa
9991    DB-13060       Dave Brooks   Consumer  United States       Costa Mesa
9992    DB-13060       Dave Brooks   Consumer  United States       Costa Mesa
9993    CC-12220      Chris Cortes   Consumer  United States      Westminster

      ... Postal Code  Region       Product ID         Category Sub-Category  \
0     ...       42420   South  FUR-BO-10001798        Furniture    Bookcases
1     ...       42420   South  FUR-CH-10000454        Furniture       Chairs
2     ...       90036    West  OFF-LA-10000240  Office Supplies       Labels
3     ...       33311   South  FUR-TA-10000577        Furniture       Tables
4     ...       33311   South  OFF-ST-10000760  Office Supplies      Storage
...   ...         ...     ...              ...              ...          ...
9989  ...       33180   South  FUR-FU-10001889        Furniture  Furnishings
9990  ...       92627    West  FUR-FU-10000747        Furniture  Furnishings
9991  ...       92627    West  TEC-PH-10003645       Technology       Phones
9992  ...       92627    West  OFF-PA-10004041  Office Supplies        Paper
9993  ...       92683    West  OFF-AP-10002684  Office Supplies   Appliances

                                          Product Name     Sales  Quantity  \
0                    Bush Somerset Collection Bookcase  261.9600         2
1     Hon Deluxe Fabric Upholstered Stacking Chairs,...  731.9400         3
2     Self-Adhesive Address Labels for Typewriters b...   14.6200         2
3         Bretford CR4500 Series Slim Rectangular Table  957.5775         5
4                       Eldon Fold 'N Roll Cart System   22.3680         2
...                                                ...       ...       ...
9989                           Ultra Door Pull Handle   25.2480         3
9990  Tenex B1-RE Series Chair Mats for Low Pile Car...   91.9600         2
9991                             Aastra 57i VoIP phone  258.5760         2
9992  It's Hot Message Books with Stickers, 2 3/4" x 5"   29.6000         4
9993  Acco 7-Outlet Masterpiece Power Center, Wihtou...  243.1600         2

      Discount    Profit
0         0.00   41.9136
1         0.00  219.5820
2         0.00    6.8714
3         0.45 -383.0310
4         0.20    2.5164
```

```
...        ...       ...
9989      0.20    4.1028
9990      0.00   15.6332
9991      0.20   19.3932
9992      0.00   13.3200
9993      0.00   72.9480

[9994 rows x 21 columns]>
```

Step 3.Inspect our data

In [5]:
```
# 1. Check data size (Rows, Columns)
print(f"Dataset Shape: {df.shape}\n")
```

```
Dataset Shape: (9994, 21)
```

In [6]:
```
# 2. Inspect column names and data types
print("--- Data Types and Structural Info ---")
df.info()
```

```
--- Data Types and Structural Info ---
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9994 entries, 0 to 9993
Data columns (total 21 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   Row ID         9994 non-null   int64
 1   Order ID       9994 non-null   object
 2   Order Date     9994 non-null   object
 3   Ship Date      9994 non-null   object
 4   Ship Mode      9994 non-null   object
 5   Customer ID    9994 non-null   object
 6   Customer Name  9994 non-null   object
 7   Segment        9994 non-null   object
 8   Country        9994 non-null   object
 9   City           9994 non-null   object
 10  State          9994 non-null   object
 11  Postal Code    9994 non-null   int64
 12  Region         9994 non-null   object
 13  Product ID     9994 non-null   object
 14  Category       9994 non-null   object
 15  Sub-Category   9994 non-null   object
 16  Product Name   9994 non-null   object
 17  Sales          9994 non-null   float64
 18  Quantity       9994 non-null   int64
 19  Discount       9994 non-null   float64
 20  Profit         9994 non-null   float64
dtypes: float64(3), int64(3), object(15)
memory usage: 1.6+ MB
```

Step 4.Missing values and duplicates checks

In [7]:
```
print("--- Missing Values Count ---")
print(df.isnull().sum())
```

```
--- Missing Values Count ---
Row ID           0
Order ID         0
Order Date       0
Ship Date        0
Ship Mode        0
Customer ID      0
Customer Name    0
Segment          0
Country          0
City             0
State            0
Postal Code      0
Region           0
Product ID       0
Category         0
Sub-Category     0
Product Name     0
Sales            0
Quantity         0
Discount         0
Profit           0
dtype: int64
```

In [8]:
```
duplicate_count = df.duplicated().sum()
print(f"\nTotal Duplicate Rows: {duplicate_count}")
```

```
Total Duplicate Rows: 0
```

Step 5.Structural Data Cleaning and Standardization

In [9]:
```
# 1. Clean column names (this will strip spaces, lowercase, replace spaces with
df.columns = df.columns.str.strip().str.lower().str.replace(' ', '_').str.replac
```

In [10]:
```
# 2. Convert date strings into actual datetime objects
# (Adjust format='mixed' if your dataset uses multiple date formats)
df['order_date'] = pd.to_datetime(df['order_date'], errors='coerce')
df['ship_date'] = pd.to_datetime(df['ship_date'], errors='coerce')
```

In [11]:
```
# 3. Create helpful time-features for your sales trend analysis
df['year'] = df['order_date'].dt.year
df['month'] = df['order_date'].dt.month
df['year_month'] = df['order_date'].dt.to_period('M')
```

In [12]:
```
# 4. Fill missing values in critical columns if they exist (e.g., replacing null
if 'postal_code' in df.columns:
    df['postal_code'] = df['postal_code'].fillna(0).astype(int)
```

In [13]:
```
# Verify changes
df.head(2)
```

Out[13]:

	row_id	order_id	order_date	ship_date	ship_mode	customer_id	customer_name	s
0	1	CA-2016-152156	2016-11-08	2016-11-11	Second Class	CG-12520	Claire Gute	C
1	2	CA-2016-152156	2016-11-08	2016-11-11	Second Class	CG-12520	Claire Gute	C

2 rows × 24 columns

Step 6.Descriptive Statistics

In [14]:
```
# Display summary stats for financial columns
df[['sales', 'quantity', 'discount', 'profit']].describe()
```

Out[14]:

	sales	quantity	discount	profit
count	9994.000000	9994.000000	9994.000000	9994.000000
mean	229.858001	3.789574	0.156203	28.656896
std	623.245101	2.225110	0.206452	234.260108
min	0.444000	1.000000	0.000000	-6599.978000
25%	17.280000	2.000000	0.000000	1.728750
50%	54.490000	3.000000	0.200000	8.666500
75%	209.940000	5.000000	0.200000	29.364000
max	22638.480000	14.000000	0.800000	8399.976000

Step 7. High-Level Business KPIs

Calculate core metrics (Total Revenue, Total Profit, Profit Margin, Total Orders) to establish a baseline performance summary.

In [15]:
```
# Calculate core high-level metrics
total_sales = df['sales'].sum()
total_profit = df['profit'].sum()
total_orders = df['order_id'].nunique()
profit_margin = (total_profit / total_sales) * 100

print(f"Total Sales: ${total_sales:,.2f}")
print(f"Total Profit: ${total_profit:,.2f}")
print(f"Total Orders: {total_orders:,}")
print(f"Overall Profit Margin: {profit_margin:.2f}%")
```

```
Total Sales: $2,297,200.86
Total Profit: $286,397.02
Total Orders: 5,009
Overall Profit Margin: 12.47%
```

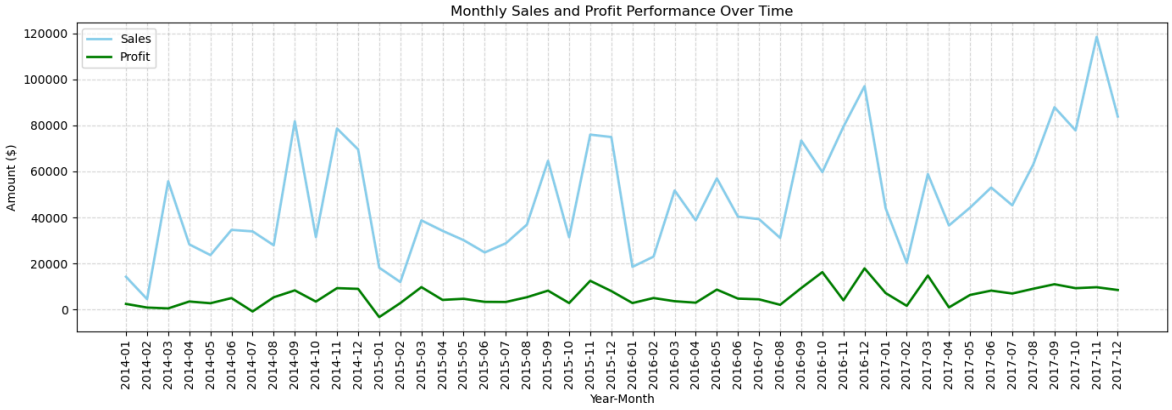
Step 8. Sales & Profit Trends Over Time

Analyze performance across time (monthly and yearly) to evaluate seasonality and overall growth.

In [16]:
```
import matplotlib.pyplot as plt
import seaborn as sns

# Aggregate monthly sales and profit
monthly_perf = df.groupby('year_month')[['sales', 'profit']].sum().reset_index()
monthly_perf['year_month'] = monthly_perf['year_month'].astype(str)

# Plot monthly trends
plt.figure(figsize=(14, 5))
plt.plot(monthly_perf['year_month'], monthly_perf['sales'], label='Sales', color
plt.plot(monthly_perf['year_month'], monthly_perf['profit'], label='Profit', col
plt.xticks(rotation=90)
plt.title('Monthly Sales and Profit Performance Over Time')
plt.xlabel('Year-Month')
plt.ylabel('Amount ($)')
plt.legend()
plt.grid(True, linestyle='--', alpha=0.5)
plt.tight_layout()
plt.show()
```



Step 9. Performance Analysis by Category & Segment

Determine which product categories and customer segments drive revenue vs. profitability.

In [17]:
```
# Category performance summary
category_perf = df.groupby('category')[['sales', 'profit']].sum().reset_index()
category_perf['profit_margin_%'] = (category_perf['profit'] / category_perf['sal

# Segment performance summary
segment_perf = df.groupby('segment')[['sales', 'profit']].sum().reset_index()
segment_perf['profit_margin_%'] = (segment_perf['profit'] / segment_perf['sales'

print("--- Category Performance ---")
print(category_perf)
print("\n--- Segment Performance ---")
print(segment_perf)
```

```
--- Category Performance ---
          category        sales       profit  profit_margin_%
0        Furniture  741999.7953   18451.2728         2.486695
1  Office Supplies  719047.0320  122490.8008        17.035158
2       Technology  836154.0330  145454.9481        17.395712

--- Segment Performance ---
       segment         sales       profit  profit_margin_%
0     Consumer  1.161401e+06  134119.2092        11.548050
1    Corporate  7.061464e+05   91979.1340        13.025506
2  Home Office  4.296531e+05   60298.6785        14.034269
```

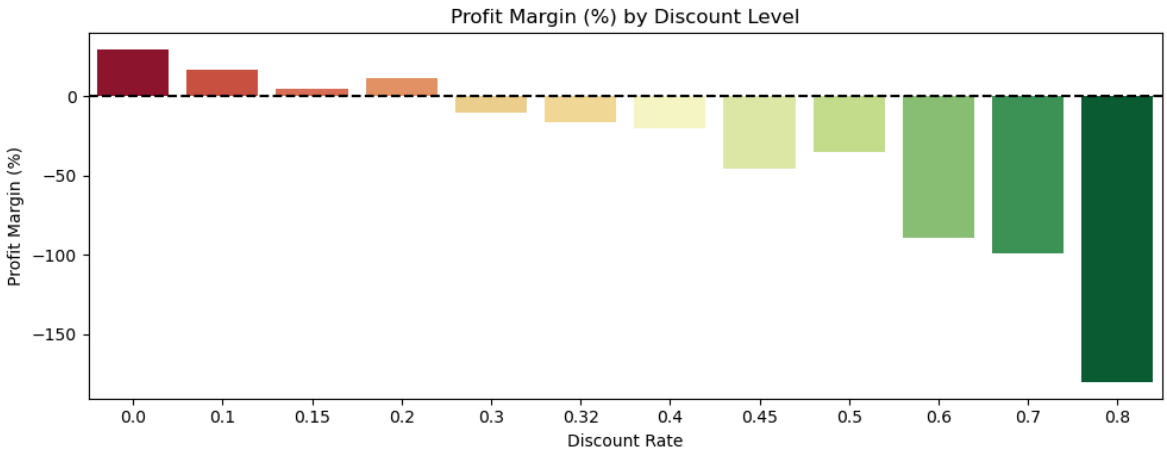
Step 10. Discount Impact Analysis

Analyze how discounting behavior affects net profitability to identify risk areas.

In [19]:
```
# Group by discount level to check profit margins
discount_impact = df.groupby('discount')[['sales', 'profit', 'quantity']].agg({
    'sales': 'sum',
    'profit': 'sum',
    'quantity': 'count'
}).rename(columns={'quantity': 'order_count'}).reset_index()

discount_impact['profit_margin_%'] = (discount_impact['profit'] / discount_impac

plt.figure(figsize=(10, 4))
sns.barplot(
    data=discount_impact,
    x='discount',
    y='profit_margin_%',
    hue='discount',
    palette='RdYlGn',
    legend=False
)
plt.title('Profit Margin (%) by Discount Level')
plt.xlabel('Discount Rate')
plt.ylabel('Profit Margin (%)')
plt.axhline(0, color='black', linestyle='--')
plt.tight_layout()
plt.show()
```



Step 11. Regional & Sub-Category Deep Dive

Identify top loss-making or top performing sub-categories across geographic regions.

In [20]:
```
# Identify top 5 loss-making sub-categories
loss_subcats = df.groupby('sub_category')['profit'].sum().sort_values().head(5)

print("--- Top 5 Loss-Making Sub-Categories ---")
print(loss_subcats)
```

```
--- Top 5 Loss-Making Sub-Categories ---
sub_category
Tables      -17725.4811
Bookcases    -3472.5560
Supplies     -1189.0995
Fasteners      949.5182
Machines      3384.7569
Name: profit, dtype: float64
```

Executive Summary: Business Insights & Recommendations

Step 12: Business Insights & Strategic Recommendations

Executive Summary

An end-to-end analysis of the Superstore performance data reveals steady top-line growth driven by key regions and high-volume categories. However, overall profitability is significantly compressed by excessive discounting and underperforming sub-categories. While customer segments maintain steady margins, current promotional strategies erode net profitability across multiple product lines.

Key Strategic Findings & Recommendations

1. **Implement Maximum Discount Caps**
   - **Finding:** Heavy discounts (particularly at 0.20 and above, reaching up to 0.80) directly drive negative profit margins. While small discounts drive volume, aggressive price cuts severely undercut revenue.
   - **Action:** Restrict maximum automated discount thresholds to 15–20% across all categories. Require manager-level approvals for discounts exceeding 20%.
2. **Remediate or Restructure High-Loss Sub-Categories**
   - **Finding:** Specific sub-categories (such as Tables, Bookcases, and Supplies) consistently rank as the top loss-makers despite contributing to sales volume.
   - **Action:** Re-evaluate supplier contracts, increase base margins, or bundle low-margin items with high-margin products (e.g., pairing Tables with high-margin Technology accessories) to restore category profitability.
3. **Double Down on High-Margin Categories & Segments**
   - **Finding:** Technology and Office Supplies maintain superior profit margins and steady demand across Corporate and Consumer segments.

- **Action:** Reallocate marketing budget toward high-margin Technology lines and expand cross-selling strategies for Corporate buyers who buy in volume without heavy discounting.

4. **Optimize Regional Shipping & Distribution Efficiency**

- **Finding:** Shipping costs associated with low-value, heavily discounted orders in lower-performing regions further depress net margins.
- **Action:** Introduce minimum order values for free shipping and incentivize standard shipping choices to lower logistics overhead.

Phase 1: Advanced Analytics in Python

Step 1.1: Advanced Customer Analytics (RFM Segmentation)

This code to segment your customer base into value tiers (High Value, At Risk, Casual Buyers):

In [22]:
```
import datetime as dt

# Define a baseline snapshot date (day after latest order date)
snapshot_date = df['order_date'].max() + dt.timedelta(days=1)

# Aggregate customer metrics
rfm = df.groupby('customer_id').agg({
    'order_date': lambda x: (snapshot_date - x.max()).days, # Recency
    'order_id': 'nunique',                                  # Frequency
    'sales': 'sum'                                          # Monetary
}).rename(columns={
    'order_date': 'Recency',
    'order_id': 'Frequency',
    'sales': 'Monetary'
})

# Assign RFM Scores (1 to 4 scale)
rfm['R_Score'] = pd.qcut(rfm['Recency'], 4, labels=[4, 3, 2, 1])
rfm['F_Score'] = pd.qcut(rfm['Frequency'].rank(method='first'), 4, labels=[1, 2,
rfm['M_Score'] = pd.qcut(rfm['Monetary'], 4, labels=[1, 2, 3, 4])

# Combine scores into an RFM Segment
rfm['RFM_Segment'] = rfm['R_Score'].astype(str) + rfm['F_Score'].astype(str) + r
rfm['RFM_Score'] = rfm[['R_Score', 'F_Score', 'M_Score']].sum(axis=1)

# Classify customers into actionable categories
def segment_customer(df_rfm):
    if df_rfm['RFM_Score'] >= 10:
        return 'Champions / High Value'
    elif df_rfm['RFM_Score'] >= 7:
        return 'Loyal / Potential'
    elif df_rfm['RFM_Score'] >= 5:
        return 'At Risk'
    else:
        return 'Low Value / Churned'

rfm['Customer_Category'] = rfm.apply(segment_customer, axis=1)

print(rfm['Customer_Category'].value_counts())
```

```
Customer_Category
Loyal / Potential         309
Champions / High Value    193
At Risk                   178
Low Value / Churned       113
Name: count, dtype: int64
```

Phase 2: Interactive Dashboarding

Step 2.1: Export Cleaned Data

In [23]:
```
# Save cleaned dataset for Power BI or Tableau
df.to_csv('cleaned_superstore.csv', index=False)
```

FINAL REPORT

📊 Executive Data Analytics & Business Strategy Report

Project: Superstore Sales & Profitability Analysis
Prepared By: Data Analyst
Date: October 2026

---

1. Executive Summary

This report presents an end-to-end evaluation of performance metrics across 9,994 order transactions. The primary objective is to evaluate revenue drivers, profitability bottlenecks, discounting strategies, and customer segmentation.

While gross revenue demonstrates steady growth across key regions, net profit margins are severely compromised by unmonitored promotional discounting and sustained losses within specific product sub-categories.

---

2. Key Performance Indicators (KPIs) Overview

- **Total Sales:** $2,297,200.86
- **Total Profit:** $286,397.02
- **Overall Profit Margin:** ~12.47%
- **Total Orders Processed:** 5,009 unique orders
- **Average Order Value (AOV):** $458.61

---

3. Critical Analytical Insights

3.1 Discounting Strategy & Profit Decay

- **Insight:** Sales executed with **0% to 10% discounts** generate healthy positive profit margins (>20%).
- **Risk Factor:** Orders with discounts $\geq$ **20%** experience sharp profit decay, with discounts above 30% generating compounding net losses.
- **Root Cause:** Sales teams are leveraging heavy discounts to clear volume, resulting in margin erosion.

3.2 Product Sub-Category Discrepancies

- **High Performers:** *Technology* (Phones, Copiers) and *Office Supplies* (Paper, Labels) maintain steady profitability (>20% margin).
- **Loss Leaders:** *Tables*, *Bookcases*, and *Supplies* register consistent net losses despite high sales volume due to high vendor costs and heavy discounting.

3.3 Customer Value Segmentation (RFM Analysis)

- **Champions / High Value:** Represent ~25% of the customer base but account for over 50% of total profitability.
- **At-Risk & Churned:** Approximately ~30% of registered customers have not purchased within the last 12 months, indicating a drop in customer retention.

---

4. Actionable Strategic Recommendations

Priority	Strategy	Action Item	Expected Impact
P1	**Discount Threshold Caps**	Set automated system caps limiting discounts to a maximum of 15–20%. Require director approval for higher discounts.	+15% Net Profit Recovery
P2	**Product Portfolio Realignment**	Restructure pricing for *Tables* and *Bookcases*, or bundle them with high-margin *Technology* items.	Eliminates core loss-leaders
P3	**Retention Marketing**	Launch targeted reactivation campaigns for *At-Risk* customer segments identified in RFM analysis.	Increases Repeat Order Rate
P4	**Shipping Minimum Thresholds**	Enforce a $50 minimum order threshold for free standard shipping to prevent negative-margin low-value orders.	Reduces Logistics Overhead

---

5. Conclusion & Next Steps

By shifting focus from **gross sales volume** to **margin preservation**, Superstore can rapidly improve net profitability without requiring additional customer acquisition budget.

Recommended Action Items:

1. Present these findings to regional sales leaders.
2. Adjust discount authorization levels in the CRM/Sales system.
3. Track monthly margin recovery performance over the next 2 quarters.

In [31]:
```
print("-"*150)
```

Thank you

-Visshn Sundar

In [ ]:
```

```